DATA 260 Homework 8

Kavan Siddesh

Section 0. Personal Configuration and Domain

GitHub repository	https://github.com/Kavan1804/data260-9486
SID4	9486
PORT_BASE	8486
PREFIX	s9486
SEED	9486
VERIFY_SEED	269486
DOMAIN_ID	6 - Rental housing listings (primary entity: Listing)
Hardware	MacBook Air with Apple M1 chip (8 GB unified memory)
Local model	llama3.2:3b (Ollama), used by the Planner, Writer and Reviewer
Kafka	apache/kafka:3.9.2 in Docker (KRaft, one broker), Python client aiokafka 0.15.0
Tagged commit hash	2a15960d37c9 (tag hw8)

Both collaborators (Sbnikitha, supriyaselvanganesan) have access to the repository. HW8 extends the same root-level application as HW5-HW7. The new code is in `kafka_service/`, `kafka_agents/`, `backend/app/routers/kafka_listings.py`, `docker-compose.yml` and `Dockerfile`; the HW8 demo, scripts and tests are in `Homework8/`, and the results (`RUN_LOG.txt`, `raw/`, `METRICS.md`, `AI_USE.md`, `verification.json`) are in `reports/hw08/`.

Part 1. Backend Service Request/Response over Kafka

My primary entity is **Listing** (rental housing listings), so the two operations are `CREATE_LISTING` and `GET_LISTING` and the id is `listing_id`. The client never calls the worker. It writes a request to `s9486_entity_requests` and waits for the answer with the same correlation_id on `s9486_entity_responses`. The worker keeps listings in a plain Python dict and makes each `listing_id` with `uuid4()`. It checks the fields with the same `ListingCreate` model my REST API has used since HW4, so a missing, empty or badly formatted field gets a clean error.

I-II. Topics and message schemas

Topic	Partitions	Written by	Read by
s9486_entity_requests	1	client (homework_demo.py) or the FastAPI wrapper	worker
s9486_entity_responses	1	worker (the topic named in reply_to)	client or FastAPI wrapper

```
Request (CREATE_LISTING):
{
  "correlation_id": "9cf1f8b7-7818-49b3-98b4-c7188cf2fe0b",
  "reply_to": "s9486_entity_responses",
  "operation": "CREATE_LISTING",
  "data": {
    "title": "Sunny 2BR near SJSU",
    "address": "125 E San Fernando St, San Jose, CA",
    "listing_code": "LST-94861",
    "available_units": 2,
    "landlord_id": 6
  }
}

Responses:
{"correlation_id": "9cf1f8b7-7818-49b3-98b4-c7188cf2fe0b", "success": true, "listing_id": "7c32b4af-
b23e-4d04-8c39-1702ab931d6c", "message": "Listing created"}
{"correlation_id": "9cf1f8b7-7818-49b3-98b4-c7188cf2fe0b", "success": true, "listing": {"listing_id":
"...", "title": "...", "...": "..."}}
{"correlation_id": "...", "success": false, "error": "Listing not found"}
```

kafka_service/worker.py, lines 36-53

```
    def handle(self, operation, data) -> dict:
        """Run one operation and return the response body (without correlation_id)."""
        if operation == messages.CREATE:
            listing, error = messages.validate_listing(data)
            if error:
                return {"success": False, "error": error}
            listing_id = str(uuid.uuid4())
            self.store[listing_id] = {"listing_id": listing_id, **listing}
            return {"success": True, "listing_id": listing_id, "message": "Listing created"}
        if operation == messages.GET:
            listing_id = data.get("listing_id") if isinstance(data, dict) else None
            if not isinstance(listing_id, str) or not listing_id.strip():
                return {"success": False, "error": "data.listing_id is required"}
            listing = self.store.get(listing_id.strip())
            if listing is None:
                return {"success": False, "error": "Listing not found"}
            return {"success": True, "listing": listing}
        return {"success": False, "error": "Unknown operation"}
```

kafka_service/messages.py, lines 29-36 - the correlation_id is created once

```
def build_request(operation: str, data: dict, correlation_id: str | None = None) -> dict:
    """The client creates the correlation_id once here; nothing downstream changes it."""
    return {
        "correlation_id": correlation_id or new_correlation_id(),
        "reply_to": config.TOPIC_RESPONSES,
        "operation": operation,
        "data": data,
    }
```

kafka_service/rpc.py, lines 73-81 - the client matches each response to its waiting request by correlation_id

```
    async def _listen(self) -> None:
        async for record in self.consumer:
            try:
                response = from_bytes(record.value)
            except ValueError:
                continue
            fut = self.pending.get(response.get("correlation_id"))
            if fut and not fut.done():
                fut.set_result((response, _meta(record)))
```

III. Docker: Kafka, worker and FastAPI

`make kafka-up` starts everything from my root `docker-compose.yml`: the Kafka broker, a one-time `kafka-init` container that creates the topics (auto-create is turned off), the worker (`python -m kafka_service.worker`) and my existing FastAPI app on 127.0.0.1:8486. The FastAPI app gets one new route file, `/api/kafka/listings`, which uses the same Kafka client and passes the caller's correlation_id through unchanged.

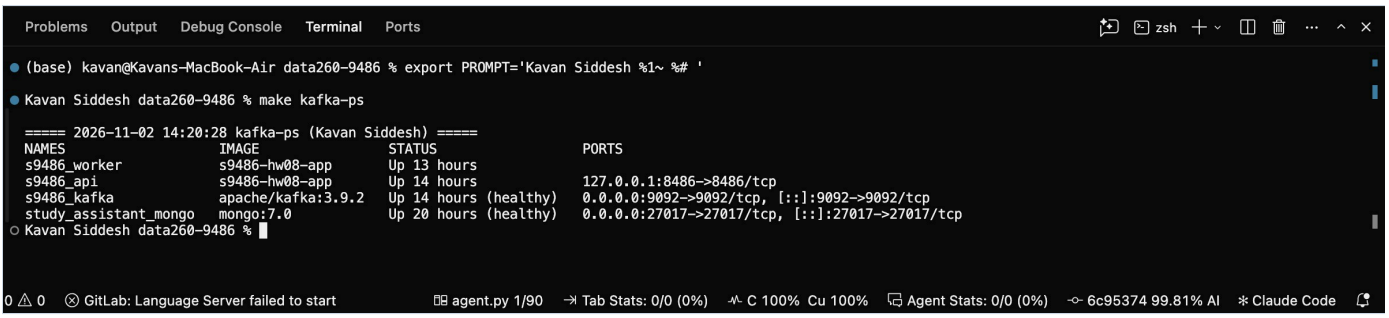


Figure 1. `make kafka-ps`: docker ps with s9486_kafka, s9486_worker and s9486_api running. (p1_docker_ps.png)



Figure 2. `make kafka-logs`: the worker prints "Consumer is ready and listening..." and FastAPI prints "FastAPI started at http://127.0.0.1:8486". (p1_service_logs.png)

IV-V. homework_demo.py: CREATE_LISTING then GET_LISTING



Figure 3. `make homework-demo`, steps 1-7: one correlation_id, the CREATE request sent to s9486_entity_requests, the worker receiving, processing and replying, and the client getting the listing_id. (p1_demo_create.png)

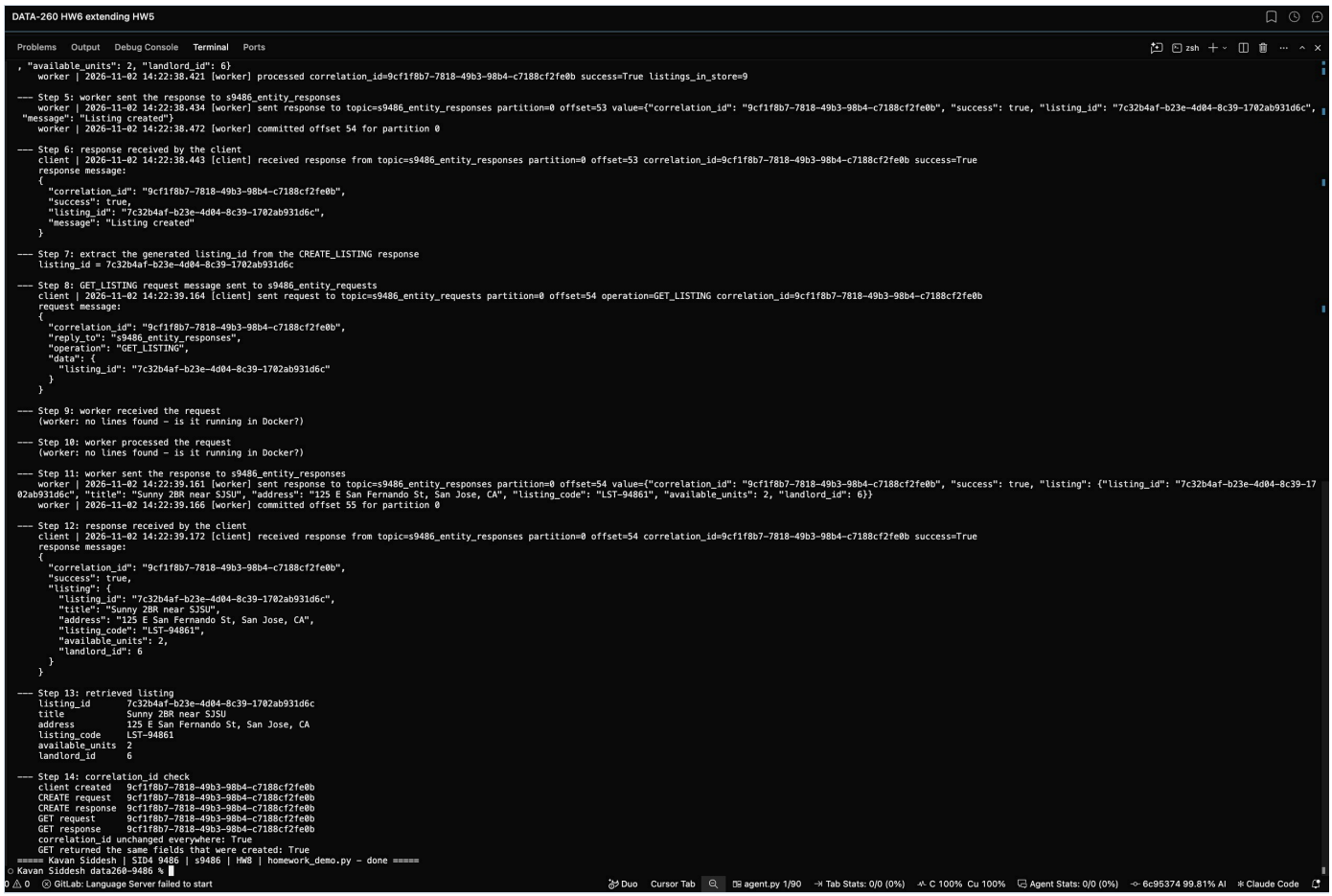


Figure 4. Steps 8-14: GET_LISTING with that listing_id, the full send/receive/process/reply/response sequence, the retrieved listing and the correlation_id check. (p1_demo_get.png)

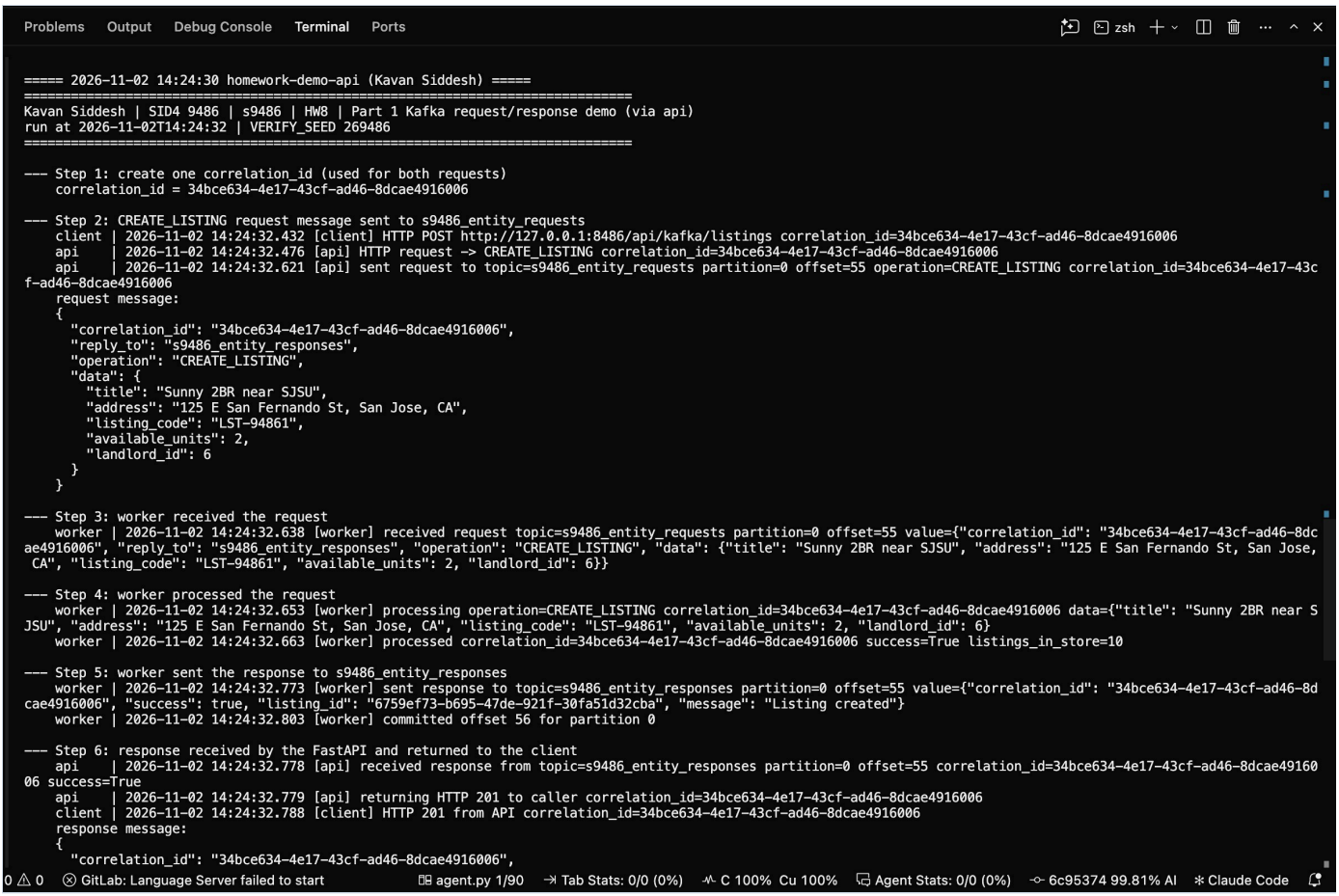


Figure 5. `make homework-demo-api`: the same flow through the FastAPI wrapper, with the API container's lines (HTTP request -> sent to Kafka -> received response -> HTTP 201/200 to the caller). (p1_demo_api.png)

Full sequence from RUN_LOG.txt (`make homework-demo`, ids shortened):

```
===== 2026-11-02 14:22:37 homework-demo (Kavan Siddesh) =====
client | 14:22:38.359 [client] sent request to topic=s9486_entity_requests partition=0 offset=53
operation=CREATE_LISTING correlation_id=9cf1f8b7...
worker | 14:22:38.389 [worker] received request topic=s9486_entity_requests partition=0 offset=53 value=
{...}
worker | 14:22:38.403 [worker] processing operation=CREATE_LISTING correlation_id=9cf1f8b7... data=
{"title": "Sunny 2BR near SJSU", "address": "12 ...
worker | 14:22:38.421 [worker] processed correlation_id=9cf1f8b7... success=True listings_in_store=9
worker | 14:22:38.434 [worker] sent response to topic=s9486_entity_responses partition=0 offset=53
value={...}
worker | 14:22:38.472 [worker] committed offset 54 for partition 0
client | 14:22:38.443 [client] received response from topic=s9486_entity_responses partition=0 offset=53
correlation_id=9cf1f8b7... success=True
client | 14:22:39.164 [client] sent request to topic=s9486_entity_requests partition=0 offset=54
operation=GET_LISTING correlation_id=9cf1f8b7...
worker | 14:22:39.161 [worker] sent response to topic=s9486_entity_responses partition=0 offset=54
value={...}
worker | 14:22:39.166 [worker] committed offset 55 for partition 0
client | 14:22:39.172 [client] received response from topic=s9486_entity_responses partition=0 offset=54
correlation_id=9cf1f8b7... success=True
correlation_id unchanged everywhere: True
```

correlation_id flow for the FastAPI run (from `raw/part1_demo_api.json` and the API container log):

Hop	correlation_id
homework_demo.py creates it and sends it in the HTTP body	34bce634-4e17-43cf-ad46-8dcae4916006
FastAPI puts it in the Kafka request (API log)	same
worker response on s9486_entity_responses	34bce634-4e17-43cf-ad46-8dcae4916006
HTTP response returned to the caller (CREATE / GET)	34bce634-4e17-43cf-ad46-8dcae4916006 / 34bce634-4e17-43cf-ad46-8dcae4916006

Results: demo runs and error cases

Run	correlation_id	listing_id	CREATE request (topic/partition/offset)	Same id	GET matches
homework_demo.py --via kafka	`9cf1f8b7-7818-49b3-98b4-c7188cf2fe0b`	`7c32b4af-b23e-4d04-8c39-1702ab931d6c`	s9486_entity_requests/0/53	True	True
homework_demo.py --via api	`34bce634-4e17-43cf-ad46-8dcae4916006`	`6759ef73-b695-47de-921f-30fa51d32cba`	s9486_entity_requests/0/55	True	True

Error case (through FastAPI 8486)	HTTP	success	error	correlation_id echoed
valid CREATE_LISTING	201	True	-	True
missing title	400	False	Invalid listing data: title: Field required	True
empty address	400	False	Invalid listing data: address: String should have at least 1 character	True
bad listing_code	400	False	Invalid listing data: listing_code: String should match pattern '^LST-\d{5}$'	True
negative available_units	400	False	Invalid listing data: available_units: Input should be greater than or equal to 0	True
data is not an object	400	False	data must be a JSON object	True
unknown listing_id	404	False	Listing not found	True
unknown operation	400	False	Unknown operation	True

8/8 cases gave the expected result and echoed the same correlation_id.

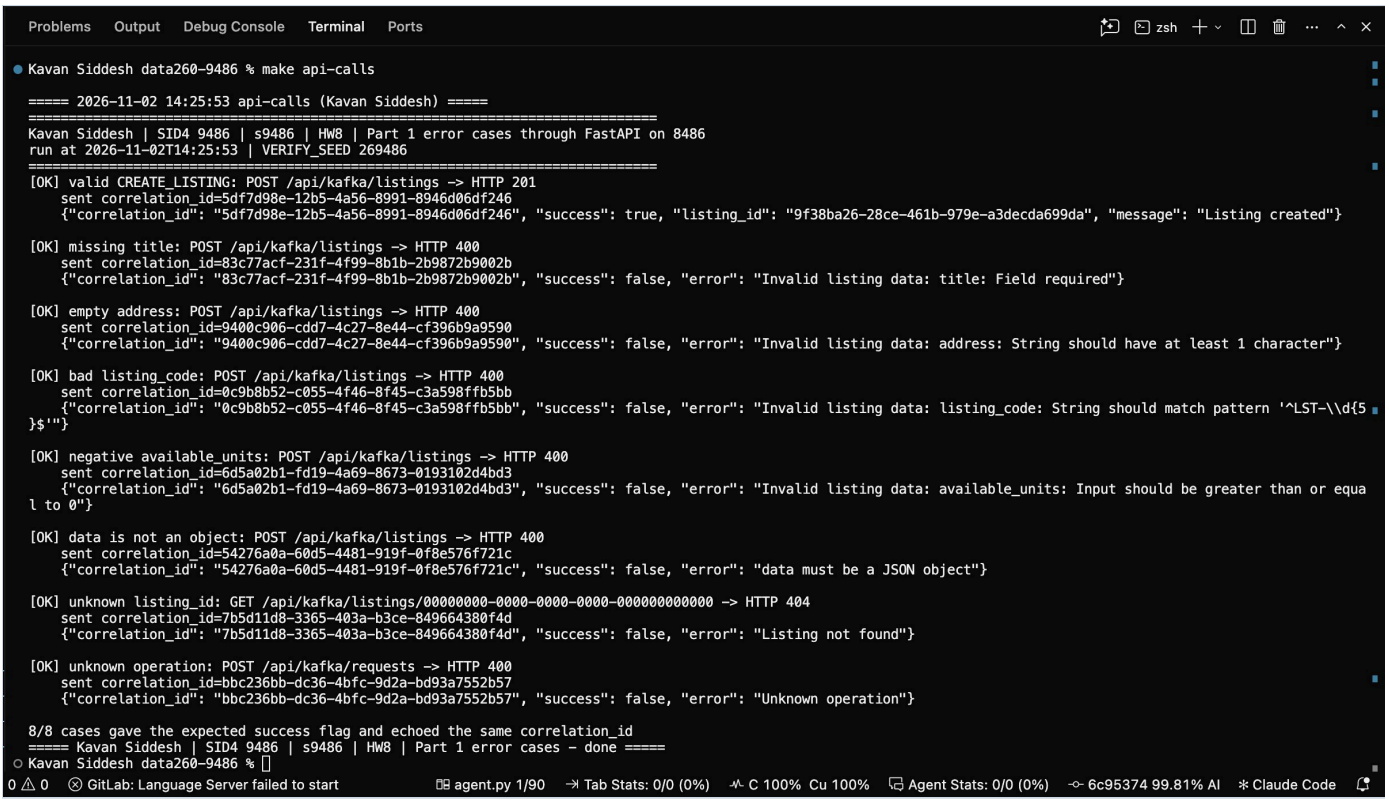


Figure 6. `make api-calls`: invalid fields, unknown listing_id and unknown operation all return success false with a clean error and the same correlation_id. (p1_errors.png)

Part 2. Three-Agent Kafka Pipeline

Planner, Writer and Reviewer are three separate Python processes. They share one small loop (`kafka_agents/base.py`): read a message, do the agent's step, send the result to the next topic, then commit the offset. No agent imports or calls another agent; the only connection between them is Kafka. All three use llama3.2:3b. The Planner returns 2-4 steps as JSON, the Writer answers the question by following the steps, and the Reviewer returns approved true/false with a short note.

Sender	Reads topic	Sends to	Message adds
send_question.py	-	s9486_inbox (1 partition)	correlation_id, question
planner.py	s9486_inbox	s9486_tasks (2 partitions)	plan
writer.py	s9486_tasks	s9486_drafts (1 partition)	draft
reviewer.py	s9486_drafts	s9486_final (1 partition)	status, answer, review

Topic creation

scripts/kafka_create_topics.sh, lines 8-19

```
create() {
  echo "creating $1 with $2 partition(s)"
  $KT --bootstrap-server "$BS" --create --if-not-exists --topic "$1" --partitions "$2" --replication-
factor 1
}

create s9486_entity_requests 1
create s9486_entity_responses 1
create s9486_inbox 1
create s9486_tasks 2
create s9486_drafts 1
create s9486_final 1
create s9486_dead_letter 1
```

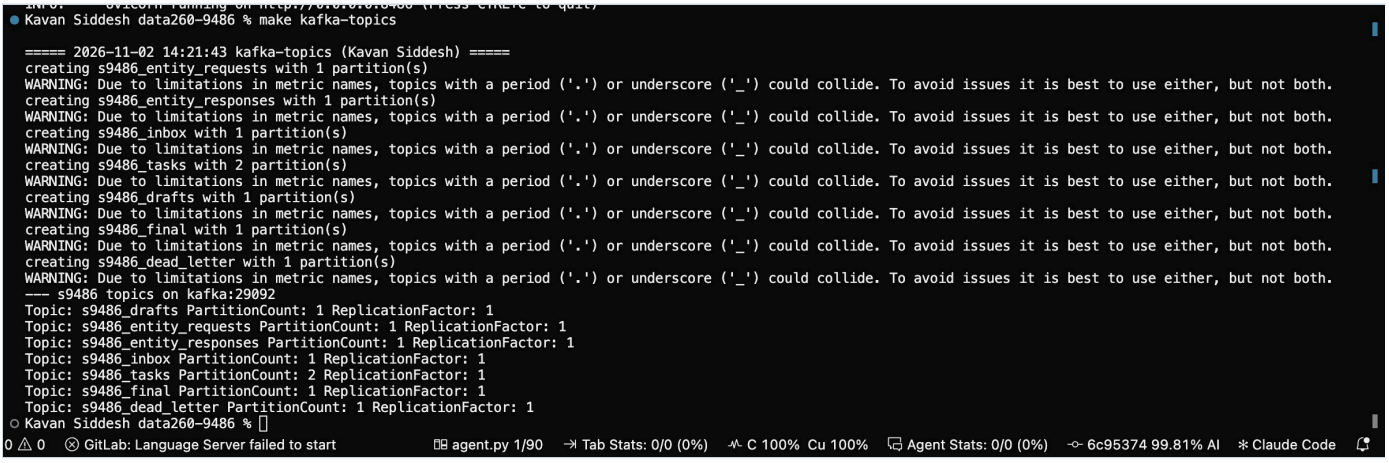


Figure 7. `make kafka-topics`: the s9486_ topics created with kafka-topics.sh, s9486_tasks with 2 partitions. (p2_topics.png)

Agent code

kafka_agents/base.py, lines 121-134 - start of the shared loop

```
    async def one_record(self, record, consumer, producer) -> None:
        a = self.args
        cid = peek_correlation_id(record.value)
        log(self.label, f"received from {where(record)} correlation_id={cid}")

        if not a.no_idempotency and cid and cid in self.done:
            log(self.label, f"duplicate correlation_id={cid}: already processed, skipping (no second
output)")
            await self.commit(consumer, record)
            return

        msg, out, error = None, None, None
        for attempt in range(1, a.max_retries + 1):
            try:
                msg = from_bytes(record.value)
```

kafka_agents/planner.py, lines 28-39

```
    def handle(self, msg: dict) -> dict:
        require(msg, "correlation_id", "question")
        if self.args.mode == "fast":
            steps, seconds = [f"Answer: {msg['question']}"], 0.0
        else:
            from kafka_agents.llm import ask

            reply, seconds = ask(SYSTEM, msg["question"], as_json=True)
            steps = reply.get("steps") if isinstance(reply, dict) else None
            if not isinstance(steps, list) or not steps or not all(isinstance(s, str) and s.strip() for
s in steps):
                raise ValueError(f"planner reply has no usable steps: {reply}")
            steps = [s.strip() for s in steps[:4]]
```

kafka_agents/reviewer.py, lines 36-44

```
            reply, seconds = ask(
                SYSTEM, f"Question: {msg['question']}\n\nPlan:\n{plan}\n\nDraft:\n{msg['draft']}",
as_json=True
            )
            if not isinstance(reply, dict) or not isinstance(reply.get("approved"), bool):
                raise ValueError(f"reviewer reply has no true/false 'approved': {reply}")
            approved, notes = reply["approved"], str(reply.get("notes", "")).strip()
        return {
            "correlation_id": msg["correlation_id"],
            "status": "approved" if approved else "needs_revision",
```

Running the agents (one terminal each)

```
make planner        # python kafka_agents/planner.py
make writer         # python kafka_agents/writer.py
make reviewer       # python kafka_agents/reviewer.py
make send-question  # python kafka_agents/send_question.py
```



Figure 8. Terminal 1: Planner. (p2_planner.png)



Figure 9. Terminal 2: Writer. (p2_writer.png)

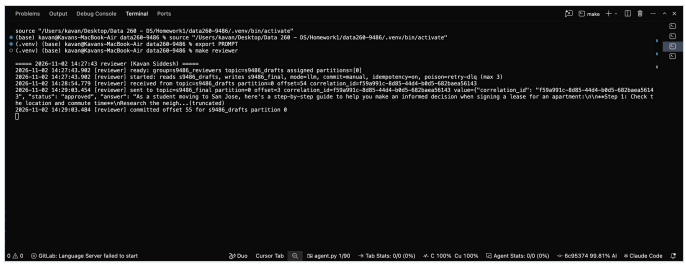


Figure 10. Terminal 3: Reviewer. (p2_reviewer.png)



Figure 11. Terminal 4: send_question.py and the final JSON. (p2_send.png)

IV. One complete message journey (inbox -> final)

From `make pipeline-demo` (all four processes started by the script, log lines with correlation_id `d17566c4-8863-45fa-a8b6-be4f3a9fc8df` in time order):

```
00:49:01.319 [send_question] sent question to topic=s9486_inbox partition=0 offset=1
correlation_id=d17566c4...
00:49:01.319 [send_question] waiting up to 400s for correlation_id=d17566c4... on s9486_final
00:49:01.321 [planner] received from topic=s9486_inbox partition=0 offset=1 correlation_id=d17566c4...
00:49:28.911 [planner] sent to topic=s9486_tasks partition=1 offset=0 correlation_id=d17566c4... value=
{...}
00:49:28.920 [writer] received from topic=s9486_tasks partition=1 offset=0 correlation_id=d17566c4...
00:49:28.932 [planner] committed offset 2 for s9486_inbox partition 0
00:49:28.945 [writer] working on correlation_id=d17566c4... with llama3.2:3b
00:50:02.373 [writer] sent to topic=s9486_drafts partition=0 offset=1 correlation_id=d17566c4... value=
{...}
00:50:02.627 [reviewer] received from topic=s9486_drafts partition=0 offset=1 correlation_id=d17566c4...
00:50:03.505 [writer] committed offset 1 for s9486_tasks partition 1
00:50:15.126 [reviewer] sent to topic=s9486_final partition=0 offset=1 correlation_id=d17566c4... value=
{...}
00:50:15.150 [send_question] received final from topic=s9486_final partition=0 offset=1
correlation_id=d17566c4...
00:50:15.166 [reviewer] committed offset 2 for s9486_drafts partition 0
```

Stage	Reads	Writes	Time (s)
planner (read s9486_inbox/0/1)	s9486_inbox	s9486_tasks	27.5
writer (read s9486_tasks/1/0)	s9486_tasks	s9486_drafts	32.7
reviewer (read s9486_drafts/0/1)	s9486_drafts	s9486_final	13.4
total (send_question -> final)			73.6

correlation_id `d17566c4-8863-45fa-a8b6-be4f3a9fc8df` kept through all stages: True. Final status `approved`, answer 161 words, model llama3.2:3b for all three agents. Reviewer notes: "The draft covers the plan and provides a clear step-by-step guide for students moving to San Jose. It emphasizes the importance of researching the location, verifying lease terms, inspecting the apartment, and reviewing the lease agreement carefully."

Final message read from `s9486_final` (`raw/final_message.json`):


```
{
  "correlation_id": "d17566c4-8863-45fa-a8b6-be4f3a9fc8df",
  "status": "approved",
  "answer": "As a student moving to San Jose, here's a step-by-step guide to help you make an informed
decision when signing a lease for an apartment:\n\n**Step 1: Check the location and commute
time**\nResearch the neighborhood and ensure it's safe and convenient for you. Consider the commute time
to school, public transportation options, and nearby amenities like grocery stores and
restaurants.\n\n**Step 2: Verify the lease terms**\nCarefully review the lease agreement to ensure you
understand the rent, utilities, and move-in/move-out dates. Ask questions if you're unsure about
anything, such as what's included in the rent and what's not.\n\n**Step 3: Inspect the
apartment**\nVisit the apartment in person to inspect for any damages or needed repairs. Take photos and
notes to document the condition of the apartment.\n\n**Step 4: Review the lease agreement
carefully**\nCarefully review the lease agreement and ask questions if you're unsure about anything.
Make sure you understand your responsibilities as a tenant and the landlord's responsibilities.",
  "review": {
    "approved": true,
    "notes": "The draft covers the plan and provides a clear step-by-step guide for students moving to
San Jose. It emphasizes the importance of researching the location, verifying lease terms, inspecting
the apartment, and reviewing the lease agreement carefully.",
    "mode": "llm",
    "model": "llama3.2:3b",
    "seconds": 12.03
  },
  "question": "I am a student moving to San Jose. What should I check before signing a lease for an
apartment?",
  "plan": [
    "Check the location and commute time to ensure it suits your needs",
    "Verify the lease terms, including rent, utilities, and move-in/move-out dates",
    "Inspect the apartment for any damages or needed repairs",
    "Review the lease agreement carefully and ask questions if you're unsure about anything"
  ],
  "writer": {
    "instance": "writer",
    "mode": "llm",
    "model": "llama3.2:3b",
    "seconds": 32.48
  },
  "trace": [
    {
```

```
      "agent": "send_question",
      "wrote": "s9486_inbox",
      "sent_at": "2026-11-02T08:49:01.302+00:00"
    },
    {
      "agent": "planner",
      "read": "s9486_inbox/0/1",
      "done_at": "2026-11-02T08:49:28.753+00:00"
    },
    {
      "agent": "writer",
      "read": "s9486_tasks/1/0",
      "done_at": "2026-11-02T08:50:01.458+00:00"
    },
    {
      "agent": "reviewer",
      "read": "s9486_drafts/0/1",
      "done_at": "2026-11-02T08:50:14.868+00:00"
    }
  ]
}
```


How Kafka helped the agents coordinate

Each agent only knows its input topic and its output topic, so I could start, stop and restart them in any order. When the Writer was slow (about 32 s per answer on my Mac), the next question would wait in `s9486_tasks` instead of blocking the Planner. Because every message keeps the correlation_id and a small `trace` list, I could follow one question through all four topics from the logs and the final JSON. Kafka also remembers each agent's committed offset, which is what Part 3 builds on.

One problem I hit: in my first logged run the Writer's 37 s model call was longer than the 10 s consumer session timeout, so Kafka dropped the Writer from its group. The draft was sent, but the commit after it failed and the Writer crashed. I raised the session timeout to 30 s and made a failed commit log a message instead of crashing (details in AI Use). The journey above is the second run, which had no errors.

Part 3. Failure Behavior

Each fix has a switch, so the same code can run with and without it. The tests use the real Kafka topics, consumer groups and commits, but the Writer runs in `--mode fast` (fixed template answer, no model call) so 20+ messages finish quickly. Each test starts its own Writer processes in a new consumer group.

Failure table

Failure scenario	Before fix	After fix
Duplicate delivery (same message sent twice)	worker: 2 different listing_ids, 2 listings for one request; writer: 2 drafts	worker: same listing_id both times, 1 listing (saved response re-sent); writer: 1 draft, duplicate skipped
Consumer killed mid-work (SIGKILL 2 s into a 6 s task)	auto-commit: offset already committed at the kill (partition=0 committed=15 log_end=15 lag=0; partition=1 committed=54 log_end=54 lag=0); restarted writer never saw task B, 0 drafts -> lost	commit after processing: partition=1 committed=55 log_end=56 lag=1 at the kill; restarted writer got task B again and finished it (1 draft, partition=1 committed=56 log_end=56 lag=0)
Poison message (not JSON / no plan)	writer crashed on the same offset twice (exit codes [1, 1]), 0/2 good tasks after it ran, partition=1 committed=57 log_end=60 lag=3	3 tries each, 2 messages moved to s9486_dead_letter, 2/2 good tasks ran, partition=1 committed=65 log_end=65 lag=0
Two partitions / two consumers (20 VERIFY_SEED messages)	no key: 4/4 listings split across partitions, 4/4 finished out of order	key = listing_code: 0/4 split, 0/4 out of order

I. Duplicate delivery

`make dup-test` sends the exact same bytes twice (same correlation_id) to the Part 1 worker and to the Writer. Without the fix the worker made a second listing and the Writer a second draft. With the fix both remember which correlation_ids they already finished. The worker sends the saved response again (same listing_id), and the Writer skips the message and only commits it.

kafka_service/worker.py, lines 74-81

```
        key = f"{cid}:{request['operation']}"
        if self.idempotent and key in self.answered:
            log(
                NAME,
                f"duplicate correlation_id={cid} operation={request['operation']}: already answered, "
                "sending the saved response again (no new listing)",
            )
            return reply_to, self.answered[key]
```

kafka_agents/base.py, lines 126-129

```
    if not a.no_idempotency and cid and cid in self.done:
        log(self.label, f"duplicate correlation_id={cid}: already processed, skipping (no second
output)")
        await self.commit(consumer, record)
        return
```

```
===== 2026-11-02 14:33:14 dup-test (Kavan Siddesh) =====
=== Worker, before fix: WORKER_IDEMPOTENCY=off
worker | 14:33:38.073 [worker] processed correlation_id=9403f747... success=True listings_in_store=1
worker | 14:33:40.084 [worker] processed correlation_id=9403f747... success=True listings_in_store=2
-> 2 responses, 2 different listing_id(s), 2 listing(s) exist for this one request
=== Worker, after fix: WORKER_IDEMPOTENCY=on
worker | 14:34:06.665 [worker] processed correlation_id=849e9c1c... success=True listings_in_store=1
worker | 14:34:08.683 [worker] duplicate correlation_id=849e9c1c... operation=CREATE_LISTING: already
answered, sending the saved response again ( ...
-> 2 responses, 1 different listing_id(s), 1 listing(s) exist for this one request
=== Writer, before fix: idempotency off
-> drafts on s9486_drafts for this correlation_id: 2
=== Writer, after fix: idempotency on
writer | 14:34:13.640 [writer] duplicate correlation_id=3a00da3a...: already processed, skipping (no
second output)
-> drafts on s9486_drafts for this correlation_id: 1
Part 1 worker  before fix idempotency=off  replay -> 2 listing(s) created
Part 1 worker  after fix  idempotency=on   replay -> 1 listing(s) created
Writer agent   before fix idempotency=off  replay -> 2 draft(s)
Writer agent   after fix  idempotency=on   replay -> 1 draft(s)
```



Figure 12. `make dup-test`: the replay before the fix (2 listings / 2 drafts) and after the fix (same listing_id / duplicate skipped). (p3_duplicate.png)

II. Consumer death

`make death-test` sends task A (finishes normally) and task B (6 s of work). Two seconds into task B the Writer is killed with SIGKILL, and a new Writer starts in the same group. With auto-commit, aiokafka had already committed B's offset in the background, so B was never done (lost). With the commit after processing, B's offset was still uncommitted (lag 1), so Kafka gave B to the restarted Writer and it finished it.

kafka_agents/base.py, lines 91-98

```
            enable_auto_commit=a.commit == "auto",
            auto_commit_interval_ms=1000,
            auto_offset_reset=a.start_from,
            # A model call can take 30-60 s on my 8 GB Mac while it swaps. With the 10 s default the
group
            # dropped the Writer mid-call, so allow 30 s without a heartbeat and 10 min between polls.
            session_timeout_ms=30000,
            heartbeat_interval_ms=5000,
            max_poll_interval_ms=600000,
```

kafka_agents/base.py, lines 181-186

```
    async def commit(self, consumer, record) -> None:
        """Manual mode: commit offset+1 for this partition, only after the output was sent (or
DLQ'd)."""
        if self.args.commit != "manual":
            return
        try:
            await consumer.commit({TopicPartition(record.topic, record.partition): record.offset + 1})
```

```
===== 2026-11-02 14:34:57 death-test (Kavan Siddesh) =====
=== before fix: writer.py --commit auto
>>> killed writer with SIGKILL while it was working on task B (exit code -9)
committed offsets right after the kill:
topic=s9486_tasks partition=0 committed=15 log_end=15 lag=0
topic=s9486_tasks partition=1 committed=54 log_end=54 lag=0
committed offsets after the restart:
topic=s9486_tasks partition=0 committed=15 log_end=15 lag=0
topic=s9486_tasks partition=1 committed=54 log_end=54 lag=0
-> drafts: task A 1, task B 0 -> task B lost (never completed)
=== after fix: writer.py --commit manual
>>> killed writer with SIGKILL while it was working on task B (exit code -9)
committed offsets right after the kill:
topic=s9486_tasks partition=0 committed=- log_end=15 lag=-
topic=s9486_tasks partition=1 committed=55 log_end=56 lag=1
writer* | 14:36:20.643 [writer-restarted] received from topic=s9486_tasks partition=1 offset=55
correlation_id=a4c2ebc7...
writer* | 14:36:26.670 [writer-restarted] committed offset 56 for s9486_tasks partition 1
committed offsets after the restart:
topic=s9486_tasks partition=1 committed=56 log_end=56 lag=0
-> drafts: task A 1, task B 1 -> task B redelivered and completed ['writer-restarted']
```



Figure 13. `make death-test`: kill during task B, committed offsets right after the kill and after the restart, for auto-commit and for commit after processing. (p3_death.png)

III. Poison message

`make poison-test` puts a bad message (not JSON, and in the second run also a task with no plan) in front of two good tasks on the same partition. Before the fix the Writer crashed on it, and every restart read the same offset and crashed again, so the good tasks were stuck behind it. After the fix each bad message is tried 3 times, then written to `s9486_dead_letter` with the error and its source offset, and committed, so the next tasks run.

kafka_agents/base.py, lines 146-157 - inside: for attempt in range(1, a.max_retries + 1)

```
            except Exception as exc:  # noqa: BLE001 - any failure counts as a failed attempt
                error = f"{type(exc).__name__}: {exc}"
                if a.poison_handling == "none":
                    # Before-fix behavior: no retry, no dead-letter, the process dies without
committing.
                    log(
                        self.label,
                        f"FAILED on offset {record.offset}: {error} -> crash, exit code 1, nothing
committed",
                    )
                    raise SystemExit(1) from exc
                log(self.label, f"attempt {attempt}/{a.max_retries} failed for offset {record.offset}:
{error}")
                if attempt < a.max_retries:
                    await asyncio.sleep(0.5 * attempt)
```

kafka_agents/base.py, lines 159-171

```
        if out is None:
            dead = {
                "correlation_id": cid,
                "failed_agent": self.label,
                "source": {"topic": record.topic, "partition": record.partition, "offset":
record.offset},
                "error": error,
                "attempts": a.max_retries,
                "raw_value": (record.value or b"").decode("utf-8", "replace")[:2000],
                "failed_at": now_iso(),
            }
            sent = await producer.send_and_wait(config.TOPIC_DEAD_LETTER, to_bytes(dead), key=(cid or
"").encode())
            log(self.label, f"gave up after {a.max_retries} attempts -> moved to {where(sent)}")
            await self.commit(consumer, record)
```

```
===== 2026-11-02 14:37:21 poison-test (Kavan Siddesh) =====
=== before fix: writer.py --poison-handling none
writer | 14:37:21.925 [writer] FAILED on offset 57: JSONDecodeError: Unterminated string starting at:
line 1 column 20 (char 19) -> crash, exit co ...
>>> writer exited with code 1
writer* | 14:37:22.135 [writer-restarted] FAILED on offset 57: JSONDecodeError: Unterminated string
starting at: line 1 column 20 (char 19) -> cra ...
>>> restarted writer exited with code 1 (same message again)
-> good tasks processed after the poison: 0/2, messages in dead-letter: 0
=== after fix: writer.py --poison-handling retry-dlq
writer | 14:37:24.246 [writer] attempt 1/3 failed for offset 61: JSONDecodeError: Unterminated string
starting at: line 1 column 20 (char 19)
writer | 14:37:24.747 [writer] attempt 2/3 failed for offset 61: JSONDecodeError: Unterminated string
starting at: line 1 column 20 (char 19)
writer | 14:37:25.748 [writer] attempt 3/3 failed for offset 61: JSONDecodeError: Unterminated string
starting at: line 1 column 20 (char 19)
writer | 14:37:25.760 [writer] gave up after 3 attempts -> moved to topic=s9486_dead_letter partition=0
offset=3
writer | 14:37:25.860 [writer] attempt 1/3 failed for offset 62: ValueError: message is missing plan
writer | 14:37:26.362 [writer] attempt 2/3 failed for offset 62: ValueError: message is missing plan
writer | 14:37:27.363 [writer] attempt 3/3 failed for offset 62: ValueError: message is missing plan
writer | 14:37:27.367 [writer] gave up after 3 attempts -> moved to topic=s9486_dead_letter partition=0
offset=4
s9486_dead_letter offset=3: error='JSONDecodeError: Unterminated string starting at: line 1 column 20
(char 19)' attempts=3 source={'topic': 's948 ...
s9486_dead_letter offset=4: error='ValueError: message is missing plan' attempts=3 source={'topic':
's9486_tasks', 'partition': 1, 'offset': 62}
-> good tasks processed after the poison: 2/2, messages in dead-letter: 2
```



Figure 14. `make poison-test`: crash loop before the fix; 3 attempts, s9486_dead_letter and the good tasks processed after the fix. (p3_poison.png)

IV. Ordering (VERIFY_SEED, 2 partitions, 2 Writers in one group)

`make ordering-test` makes 20 related messages with `random.Random(269486)`: 4 listings with 5 updates each (seq 1-5), mixed into one send order. Two Writers join one consumer group, so each owns one partition of `s9486_tasks`. The same 20 messages are sent twice: once with no key, and once with the listing_code as the key. Each Writer logs every message it finished (`raw/ordering_*.csv`).

Homework8/scripts/failure_ordering.py, lines 106-107 - the partition key

```
        items = [((m["test"]["entity"].encode() if keyed else None), to_bytes(m)) for m in msgs]
        metas = send_raw(config.TOPIC_TASKS, items)
```

Run	Listing	Partitions	Writers	Processed seq order	In order
unkeyed	LST-54549	[0, 1]	writer-1, writer-2	[1, 2, 4, 3, 5]	False
unkeyed	LST-62000	[0, 1]	writer-1, writer-2	[1, 4, 2, 3, 5]	False
unkeyed	LST-90996	[0, 1]	writer-1, writer-2	[1, 3, 2, 4, 5]	False
unkeyed	LST-98538	[0, 1]	writer-1, writer-2	[1, 3, 2, 4, 5]	False
keyed	LST-54549	[1]	writer-1	[1, 2, 3, 4, 5]	True
keyed	LST-62000	[1]	writer-1	[1, 2, 3, 4, 5]	True
keyed	LST-90996	[1]	writer-1	[1, 2, 3, 4, 5]	True
keyed	LST-98538	[1]	writer-1	[1, 2, 3, 4, 5]	True

```
===== 2026-11-02 14:37:52 ordering-test (Kavan Siddesh) =====
send order: 62000#1 54549#1 98538#1 62000#2 90996#1 62000#3 98538#2 54549#2 54549#3 62000#4 54549#4
54549#5 62000#5 98538#3 98538#4 90996#2 98538#5 90996#3 90996#4 90996#5
consumer group s9486_writers_order_unkeyed_143752: writer-1 has partition [0], writer-2 has partition
[1]
processing order: 54549#1(p1) 90996#1(p1) 54549#2(p1) 62000#1(p0) 98538#1(p0) 62000#4(p1) 62000#2(p0)
62000#3(p0) 54549#4(p1) 98538#3(p1) 98538#2(p0) 98538#4(p1) 54549#3(p0) 90996#3(p1) 54549#5(p0)
62000#5(p0) 90996#2(p0) 98538#5(p0) 90996#4(p0) 90996#5(p0)
-> unkeyed: 20/20 processed, 4 listing(s) split across partitions, 4 listing(s) out of order
consumer group s9486_writers_order_keyed_143759: writer-1 has partition [1], writer-2 has partition [0]
processing order: 62000#1(p1) 54549#1(p1) 98538#1(p1) 62000#2(p1) 90996#1(p1) 62000#3(p1) 98538#2(p1)
54549#2(p1) 54549#3(p1) 62000#4(p1) 54549#4(p1) 54549#5(p1) 62000#5(p1) 98538#3(p1) 98538#4(p1)
90996#2(p1) 98538#5(p1) 90996#3(p1) 90996#4(p1) 90996#5(p1)
-> keyed: 20/20 processed, 0 listing(s) split across partitions, 0 listing(s) out of order
```

Without a key, aiokafka picks a random partition for each message, so 4 of 4 listings had their updates split across both Writers and 4 of 4 finished out of order (a later seq done before an earlier one). With the key, every update of a listing went to the same partition and was handled by one Writer in order. All 4 keys hashed to partition 1, so one Writer did all the work and the other sat idle. A key keeps the order, but with only a few keys the load can be uneven.

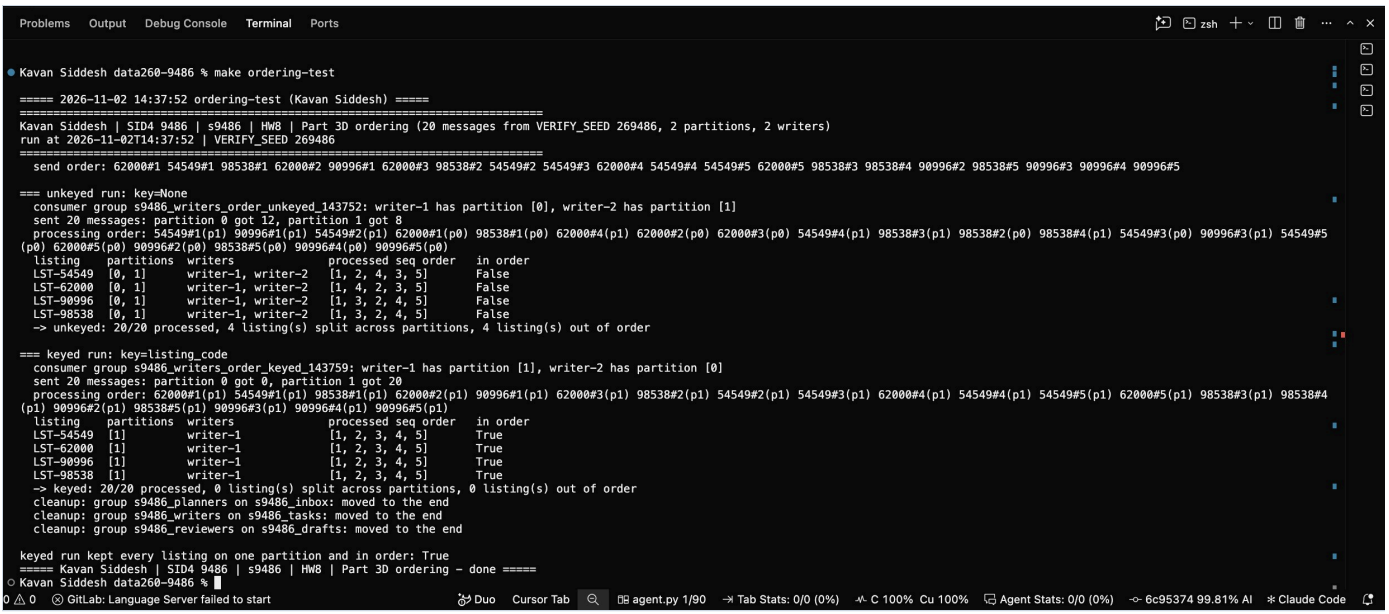


Figure 15. `make ordering-test`: both runs, partition per listing and processed order. (p3_ordering.png)

V. Comparison with my HW2 in-process graph

In HW2 my LangGraph workflow (supervisor -> planner -> reviewer) ran as function calls in one Python process. Each node got the previous node's state as a return value. Here the same kind of hand-off goes through Kafka topics between separate processes.

	What changed
Benefit 1: failures stay separate	When the Writer crashed or was killed, the Planner and Reviewer kept running and the work waited in s9486_tasks. In HW2 one exception ended the whole graph run.
Benefit 2: independent scaling	I could run two Writers in one consumer group and Kafka split the 2 partitions between them, without changing the Planner or Reviewer. In HW2 every step ran one after another in the same process.
Responsibility 1: duplicates and offsets	Kafka can deliver a message again (a replay, or a crash before the commit), so I had to add the idempotency check and decide when to commit. A function call in HW2 ran exactly once.
Responsibility 2: ordering and bad messages	Order is only kept inside one partition, so I had to choose a key. A bad message can block a partition forever, so I had to add retries and a dead-letter topic. In HW2 the order was just the order of the code.

Verification and Conclusions

`make verify-hw08` writes `reports/hw08/verification.json`. Last run: 19/19 checks passed (commit `2a15960d37c9`, model llama3.2:3b, SEED 9486, VERIFY_SEED 269486). It checks behavior, not wording.

Check	Result
config_values	PASS
docker_containers_running	PASS
kafka_reachable	PASS
topics_created	PASS
create_listing_ok	PASS
listing_id_present	PASS
get_listing_ok	PASS
correlation_id_preserved	PASS
unknown_operation_error	PASS
unknown_listing_id_error	PASS
invalid_fields_rejected	PASS
duplicate_no_side_effect	PASS
fastapi_wrapper_8486	PASS
pipeline_reaches_final	PASS
poison_reaches_dead_letter	PASS
ordering_files_exist	PASS
offline_tests_pass	PASS
ruff_clean	PASS
evidence_files_exist	PASS

Conclusions

- Splitting the Listing service into a client and a worker worked as long as the correlation_id is created once and copied unchanged; both demo runs and all error cases kept it.
- The three agents only talk through Kafka. One question took about 74 s end to end, almost all of it the local model.
- Each failure was easy to cause once the parts were separate processes: a replay made a second listing, auto-commit lost a task when the Writer died, and one bad message blocked a partition. Idempotency, committing after processing, and retries + a dead-letter topic fixed them in my tests.

- Keys keep related messages in order, but they decide the load too: my 4 keys all landed on one partition.
- The JMeter instructions are a separate DATA 236 demo (`/users-direct` on port 8000) and are not part of the HW8 PDF, so I did not run a load test.

AI Use

1. What I used an AI assistant for, and what I did myself.

I used an AI assistant (Claude Code) for most of this homework, the same way as in HW7. It read the HW8 PDF, my HW7 code and the two demo ZIPs with me, and pointed out where the demos did not match the assignment (the topic names, the message shape, auto-commit, port 8000, OpenAI in the multi-agent demo). It wrote most of the HW8 code from my instructions: the Kafka worker and RPC client, the FastAPI wrapper route, the docker-compose file and Dockerfile, the Planner/Writer/Reviewer agents and their shared loop, the failure test scripts, the offline tests and verify_hw08.py. It also helped with the Markdown files and the report, explained error messages, and debugged the problems below. Claude Code ran the logged runs in RUN_LOG.txt in my terminal on my Mac, and I approved each command.

What I did myself: I chose the design choices it asked me about (show duplicate delivery on the Part 1 worker, use a fast template mode for the failure tests, run FastAPI in Docker, keep short code snippets in the report), started Docker and Ollama, read the logs and the raw files to check that they show what the report says, took the screenshots, and decided what to submit.

2. One AI-produced output that was wrong or unsuitable.

The first version of the agent loop that the AI wrote used aiokafka's default consumer settings (10 s session timeout) and did not handle a failed offset commit. That was fine for the fast-mode tests, but in my first real pipeline run the Writer's model call took 37 s. Kafka dropped the Writer from its consumer group during the call, and when the Writer tried to commit after sending its draft, aiokafka raised `CommitFailedError` and the Writer process crashed.

3. How the problem was detected.

The pipeline still reached s9486_final, so the summary looked fine. I only saw the problem in the Writer's log (`raw/agent_logs/writer.log`) and in RUN_LOG.txt: "OffsetCommit failed ... UnknownMemberIdError", then a traceback ending in `CommitFailedError`, right after the "sent to s9486_drafts" line. The first test run of the day also showed many "Heartbeat failed" warnings, and the Mac was using about 12 GB of swap.

4. What was changed and why it works now.

The agents now allow 30 s without a heartbeat (`session_timeout_ms=30000`) and 10 minutes between polls, so a slow model call does not get the Writer kicked out. If a commit still fails because the group rebalanced, the loop logs it and keeps going instead of crashing. The output was already sent, so Kafka will deliver the message again and the duplicate check (correlation_id already done) skips it. I also lowered the agents' model context to 2048 tokens and gave the Kafka broker a smaller heap so the Mac swaps less. After the change the second pipeline run committed every offset with no errors, and an offline test (`test_commit_failure_after_rebalance_does_not_crash`) checks the new path.